Contents

Data construction and descriptive statistics **2**
Contents . . . 2
1. Know the unit of every table . . . 2
2. Conversion and concatenation . . . 3
3. Merging the two source families . . . 3
4. Calendar matching and `master_dat.py` . . . 4
5. `summ_stats.py`: projection and sample definitions . . . 5
6. Household summaries and demographics . . . 5
7. Flavor recovery and two taxonomies . . . 6
8. Descriptive choice dynamics . . . 6
9. Why the summary code writes temporary partitions . . . 8
10. Running, outputs, and verification . . . 8

Data construction and descriptive statistics

This chapter explains the scripts in `cleaning_merging/`, the summary pipeline in `summ_stats.py`, and the flavor-recovery utility. It describes current source behavior, including consequential assumptions in older scripts. Continue with the estimation chapter after the prepared purchase table is built.

Contents

1. Data units and the overall pipeline
2. Conversion and concatenation
3. Panel and retail merges
4. Calendar matching and the yogurt sample
5. Summary input projection and sample restrictions
6. Household demographics and participation
7. Flavor recovery and classification
8. Shares, concentration, transitions, and spells
9. Memory management
10. Outputs, commands, and interpretation

1. Know the unit of every table

The data pipeline combines household purchase records with store-level retail prices. These sources answer different questions. Household records say who bought which UPC and how many packages. Retail records supply a store/week/UPC price. Joining them creates a purchase line with a matched price; it does not create a complete retail assortment panel.

Object	Intended unit	Important key
Purchases	A recorded UPC purchase line	Trip ID and UPC; multiple lines may exist
Trips	A shopping trip	`trip_code_uc`
Panelists	Household demographics in a panel year	Household, panel year
Panel products	UPC metadata	UPC
Retail movement	Store/UPC/week record	Store, UPC, week end
Retail stores	Annual store metadata	Store, year
Extra products	Annual UPC descriptors	UPC, year
Summary household table	One retained household	Household
Sparse flavor quantities	Household/week/flavor	Household, Saturday week, flavor
Estimation input	Original retained purchase line plus resolved label	Original fields retained

These are intended keys. Older merge scripts do not enforce every uniqueness condition with join validation. A

left join can multiply rows if the right side has duplicate keys. A successful run and a nonempty output therefore do not prove that the joins are correct.

The logical execution order is panel conversion → panel concatenation → panel merge → panel week attachment; independently, retail conversion → retail concatenation → retail merge; then master matching → summaries → estimation. Paths in the older scripts are hard-coded to `/scratch/dtm63837/Kilts_Panel`. The summary script exposes paths through command-line arguments.

2. Conversion and concatenation

2.1 `panel_convert.py`

This reads annual panelists, purchases, and trips and selected master files, converts TSV to Parquet, lowercases names, then creates narrower `cleaned_` files. TSV scanning uses tab separators, disables quote parsing, and infers types from a prefix. Some master reads use lossy UTF-8 decoding.

The purchase projection retains trip ID, UPC, quantity, total price paid, and deal flag. Trip projection supplies household, purchase date, store, and year. Panelists provide income, household size, head ages/employment, race, Hispanic origin, and DMA. Product metadata supplies module, group, brand, and size fields.

Projection reduces subsequent I/O, but a column dropped here cannot be recovered from the cleaned file. A new downstream analysis needing another raw variable must add it to the projection and rebuild the affected stages.

2.2 `retail_convert.py`

Annual sources include `products_extra`, `rms_versions`, stores, and movement modules 3603 and 3612. Master sources include products and brand variations. The current extraction loop covers 2006–2015. Movement projections keep store, UPC, week end, and price; they do not retain every original retail field.

The cleaner matches table names by longest prefix first, so `products_extra` does not accidentally receive the shorter `products` projection. It skips files already named `cleaned_`. Conversion loops report missing files; those messages must be checked because a loop can continue with incomplete annual coverage.

2.3 Concatenating years

`panel_concat.py` scans cleaned annual files into `combined/panelists.parquet`, `combined/purchases.parquet`, and `combined/trips.parquet`. `retail_concat.py` builds combined store, extra-product, module, and RMS-version tables. It explicitly casts common schema-conflict fields before `vertical_relaxed` concatenation: flavor/size codes become strings, UPC/store IDs Int64, and panel year Int32.

Concatenation stacks rows; it does not establish key uniqueness or remove duplicate years. Wildcard-selected files are part of the effective input. A stale annual file in the directory can silently become part of a rebuild. `panel_dat.py` and `retail_dat.py` are empty placeholders in this checkout.

3. Merging the two source families

3.1 `panel_merge.py`: purchases are the anchor

The merge chain is

```
purchases INNER JOIN trips ON trip_code_uc
          LEFT JOIN panelists ON household_code, panel_year
          LEFT JOIN products ON upc
    -> Data/merged_panel.parquet
```

Join keys are cast consistently before merging. The inner trip join can discard unmatched purchase lines; the demographic/product joins are intended to retain lines while adding missing metadata where no match exists.

The script reports row count, count relative to purchases, key nulls, file size, and columns. It raises on an empty result. However, 100% overall row retention can hide offsetting losses and duplication; verify right-key uniqueness

and unmatched keys when auditing a production rebuild. The printed claim that the file is ready is a status message, not a full statistical validation.

3.2 `retail_merge.py`: movements are the anchor

Retail movements from modules 3612 and 3603 are concatenated. Calendar year is derived from the integer week end using `week_end // 10000`. Then metadata is attached through left joins:

1. RMS versions, deduplicated to the first row per UPC, supply `upc_ver_uc`.
2. Products match UPC and version, deduplicated on that composite key.
3. Stores match store and calendar year, retaining the first duplicate.
4. Extra products match UPC and year, prioritizing nonnull flavor descriptions through a sort before retaining one row.
5. Brand variations match the first row per brand code.

Output is `Data/RMS/merged_rms.parquet`. Selecting a first row prevents many joins from multiplying records, but it is also a substantive reconciliation rule. For example, first-per-UPC RMS version selection discards version/year variation. The source does not establish that the chosen row is economically preferred.

Post-merge diagnostics compare movement and output row counts and report nonnull metadata rates. A nonnull brand-code field is not always evidence of a successful brand-dictionary match, because the code may inspect the original join key. Read each diagnostic as the particular column check it implements.

4. Calendar matching and `master_dat.py`

4.1 Checking the upstream Sunday convention

`panel_week.py` reads `merged_panel.parquet`, adds integer `week_end`, writes a temporary replacement, and replaces the original file with `os.replace`. Its implemented day offset is

$$6 - (\text{weekday}(d) \bmod 7).$$

With Polars Monday=1 through Sunday=7, Monday advances five days and Saturday zero. Sunday has remainder zero, giving an offset of six days. This differs from a direct `6-weekday` expression, which would send Sunday backward. The exact source formula must be evaluated with the installed Polars weekday convention before attributing lost Sunday rows.

The current summary and estimation formula is $(6 - \text{weekday}(d)) \bmod 7$; under the documented 1–7 convention, it agrees with the upstream formula for all weekdays. Older handoff warnings about Sunday loss should therefore be treated as an audit question about the code and data version actually used to build the input, rather than proof of a bug in the checked-in formula.

Week end is a matching key, so a historical erroneous build can still matter: later recomputation does not recover rows discarded by an earlier inner join. For a rebuild, explicitly check Sunday, Saturday, year-end, and leap-year dates against the retail calendar, and compare the saved `week_end` to recomputation.

4.2 Master matching

`master_dat.py` reads `Data/merged_rms.parquet`, while `retail_merge.py` writes `Data/RMS/merged_rms.parquet`. This is a real path mismatch in the current scripts. A prepared server file or manual copy may have bridged it; the source does not do so automatically. Resolve the intended path before a fresh run.

The master script projects retail columns to year/week/UPC/store, module code, and price; filters module codes 3603 and 3612; and keeps the first quote for each composite key. It collects this reduced retail table into memory, then processes panel purchases one year at a time.

For each year, it inner joins on

$$(\text{panel_year}, \text{week_end}, \text{upc}, \text{store_code_uc}).$$

Only purchases with matching retail keys survive. This selects on retail coverage, calendar alignment, UPC coding, and store coding. The result is not every household yogurt purchase. After yearly outputs are concatenated into `Data/yogurt_estimation_sample.parquet`, temporary year files are removed.

Because the match includes both panel year and calendar week end, year-boundary conventions require attention: a late-December purchase whose Saturday falls in January can have a panel year different from the retail-derived year. That possibility should be audited in the private data.

The source deduplicates retail keys by keeping the first quote rather than checking for contradictory prices. Estimation checks conflicts in its merged input, but cannot detect contradictions already discarded upstream.

The master audit reports entity counts, key nulls, years, price distribution, and file size. It does not report all purchase losses against the original yogurt universe or establish that all surviving prices represent the desired package/offer convention.

5. `summ_stats.py`: projection and sample definitions

`scan_purchases` starts from a lazy Parquet scan. It requires household, year, purchase date, trip, UPC, and quantity. It projects optional demographics into a common schema; absent columns become null strings. Income can originate as `household_income` or `income`; Hispanic origin can originate as `hispanic_origin` or `hispanic`.

Dates are parsed from their string prefix and quantities become Float64. Numeric-looking UPCs are canonicalized as strings, removing leading zeros and trailing `.0` without passing through floating point. Price uses `price`, then `price_rms`, or null if neither exists. This projection is deliberately narrower than the final estimator input: the later export reopens the original file to preserve store IDs and all other purchase columns.

`run` maintains two populations. `full_panel` uses `merged_panel.parquet` in the selected years and does not impose matched-price or repeated-yogurt rules. `matched_yogurt` starts from the master sample, removes missing/nonfinite/nonpositive prices, and requires two distinct Saturday-ending purchase weeks. The two-week rule is implemented by comparing a household's minimum and maximum week; multiple trips or lines in one week still count as one week.

The saved descriptive matched sample precedes estimator menu and IV exclusions. It can include a household later dropped for fewer than two usable likelihood weeks. Full-panel summaries cover households represented in purchase records, not every potential household in an enrollment roster.

6. Household summaries and demographics

`household_summary` produces one row per household, so prolific shoppers do not receive extra weight in demographic tables. Counts include distinct panel years, first/last year, distinct trips, shopping dates, distinct UPCs, purchase lines, and total packages. Trips use `trip_code_uc`; shopping days use purchase date. They need not agree because two trips can occur on the same day.

Demographic history is projected, deduplicated, and stably sorted by household, year, and date. Each field takes its earliest nonmissing value independently. Consequently, the resulting household row need not correspond to a single original date if fields became available at different times. Missing means null; unusual nonnull source codes need their own codebook interpretation.

Zero male/female head-age codes become missing. Combined `head_age` prefers female head age when available, then male head age. This is a category-selection rule, not an average of the heads' ages. Hispanic origin string `"1"` overrides the main race category to `"Hispanic"`; `race_original` preserves the prior value.

Approximate exposure is

$$W_i^{approx} = Y_i^{observed} \times 365.25/7, \qquad \text{trip rate}_i = T_i / W_i^{approx}.$$

A household observed in 2006 and 2008 has two observed years, not three. This avoids filling a missing year, but assumes full-year exposure in each observed year. It is not verified enrollment time. In the matched sample, trips count trips represented by retained yogurt purchases, not all shopping.

`demographic_tables` reports categories including an explicit `Missing` group, using all households in the denominator. Tables are unweighted. Label dictionaries for race, income, Hispanic origin, and age are initially empty; populate them only with verified codebook mappings. Numeric income/head-age means in the overview are means of source category codes, not dollars or years.

`save_table` prints each table and writes paired CSV and LaTeX files. The LaTeX export escapes source text and uses two decimal places. Year coverage is computed one year at a time from purchases, retaining exact distinct trip and household counts rather than summing household averages.

7. Flavor recovery and two taxonomies

7.1 Supplementing the reference

`recover_flavor_reference` normalizes UPCs and anti-joins annual product metadata against existing reference UPCs. Only UPCs absent from the reference are added; it does not overwrite a bad or blank existing descriptor. Candidate additions must have a nonmissing usable flavor description. When annual years exist, the earliest candidate metadata is selected per UPC; without years, the first candidate is kept.

The supplementary rows are saved separately beside the combined recovered reference. The function refuses to overwrite the original reference. `run` uses this automatically if the annual product source exists; otherwise it logs that only the supplied CSV will be used. `recover_flavors.py` exposes recovery as a standalone command with `--flavors`, `--products`, and `--output`.

`valid_flavor_labels` is a small descriptor-validity helper. The active recovery code also expresses validity filters directly in its lazy query; the helper's presence should not be read as an additional transformation applied everywhere.

7.2 Detailed versus grouped labels

`flavor_reference(..., detailed=True)` lowercases and trims `flavor_descr`, collapses whitespace, maps blank descriptions to `unknown`, and maps `unflavored` to `plain`. These detailed labels drive the main summary directory.

The grouped pass searches combined flavor and optional product descriptions: plain/unflavored first, then berry/straw/blue/rasp/black/cran/cherry/wildberry, and otherwise other. This broad text rule is reproducible, but it is not a hand-verified flavor ontology. "Cherry" enters berry by construction, and a description matching both plain and berry enters plain.

Duplicates are reconciled by the first row per normalized UPC. Conflicting classifications are counted and optionally exported. Missing descriptors on a known UPC still classify: `unknown` in detailed labels, usually `other` in groups. An entirely unmatched purchased UPC causes `yogurt_weeks` to export an unmatched UPC audit and raise, rather than silently deleting the purchase.

The grouped pass writes `estimation_flavor_reference.csv` and embeds its labels in the final prepared purchase file. This avoids reclassification during the normal estimation path.

8. Descriptive choice dynamics

8.1 Sparse weekly quantities

`yogurt_weeks` joins the reference and aggregates quantity by household, Saturday week, and flavor. Only observed positive quantities occupy rows. It audits unmatched labels, invalid quantities, and bad prices. Nonpositive/nonfinite quantities stop processing; the summary stage does not enforce integer quantity as strictly as the estimator does.

Keeping the table long avoids creating a dense household-week-by-detailed-flavor matrix. No unobserved weeks or zero-purchase rows are inserted.

8.2 Aggregate shares and household concentration

The aggregate package share is

$$\text{Share}_j = 100\frac{\sum_{it} q_{ijt}}{\sum_{itk} q_{ikt}}.$$

It weights heavy purchasers more than light purchasers. The saved share table omits flavors rounding to zero at two displayed decimals while retaining the original denominator; displayed shares can therefore sum slightly below 100.

For household totals $Q_{ij} = \sum_t q_{ijt}$, define

$$s_{ij} = Q_{ij} / \sum_k Q_{ik}, \qquad HHI_i = \sum_j s_{ij}^2, \qquad L_i = \max_j s_{ij}.$$

`yogurt_household_flavors.csv` stores number of flavors bought, HHI, and largest share. With counts 6,3,1, HHI is .36+.09+.01=.46 and largest share .6. With two equally purchased flavors HHI=.5; with one flavor HHI=1. It is on a 0–1 scale.

Detailed and grouped HHI answer different questions. Combining categories adds cross terms: merging a and b raises HHI by $2s_a s_b$. Thus grouped HHI is at least detailed HHI for the same purchases. The mean household HHI weights each household equally, unlike the aggregate package shares.

`make_hhi_figures` uses all household values, with 40 equal-width bins from zero to one for the histogram and an empirical step CDF. Histogram weights sum to 100%; these are percentages, not density units. The CDF gives the share of households with HHI at most a threshold. Neither is truncated to 20 flavors.

8.3 Transition probabilities

`flavor_summaries` builds one basket per household-week. A basket has a `single_flavor` label only when it contains exactly one flavor. A pair is eligible only when current and immediately preceding observed baskets are both single-flavor and exactly seven days apart. Mixed baskets and gaps break pairs.

For transition counts n_{ab}, rows are previous flavor a, columns current flavor b, and

$$\widehat{P}(b \mid a, \text{eligible}) = n_{ab} / \sum_k n_{ak}.$$

Stays are included in the denominator. A row with no eligible transitions has undefined probabilities, represented as missing. The switch rate is the percentage of eligible pairs with different flavors; it is not the share of all households that ever switch.

For example, berry on Jan 2, berry on Jan 9, plain on Jan 16, and berry on Jan 30 produces berry→berry and berry→plain. Jan 30 contributes no transition because Jan 23 is missing. Berry's observed transition probabilities are .5 stay and .5 plain. This describes adjacent single-flavor purchase behavior, not all consumption transitions.

8.4 Spells and graphs

A spell continues only across eligible same-flavor adjacent weeks. Changing flavor, encountering a mixed basket, crossing a gap, or starting another household creates a new spell identifier. Mixed baskets are excluded from single-flavor spell output. Spell length counts consecutive observed calendar weeks, not packages, days, or elapsed time between arbitrary purchases.

The mean spell length averages spells; a household with many spells contributes many observations. `yogurt_dynamics` also reports mixed-basket frequency, weeks above seven packages, eligible pairs, and the median observed week gap. The above-seven measure diagnoses the optional estimator capacity convention; the summary code does not discard these baskets.

Flavor-share and transition figures show up to the 20 most purchased flavors. Transition probabilities are calculated before subsetting, so displayed rows need not sum to one if destinations lie outside the plotted subset. Saved transition tables retain the fuller taxonomy.

9. Why the summary code writes temporary partitions

Lazy scans defer work; they do not guarantee constant memory. Exact distinct counts and group aggregations retain state, and repeated scans of a giant purchase source are expensive. `partition_purchases` hashes household IDs with seed zero into `hash(id) % batches`, then writes physical Parquet partitions. All rows for a household remain together, preserving history and exact distinct counts across years.

`household_batches` reads one partition at a time, builds household summaries, and writes small intermediate files. Those same purchase partitions are reused for detailed and grouped weekly flavor aggregation. Household demographic history is narrowed and deduplicated before sorting once across all fields; exact purchase totals are computed separately from all lines.

This avoids rescanning the original source once for every household batch and avoids a dense detailed-flavor pivot. It still needs temporary disk space and can retain large aggregation state within a partition. Increasing `--household-batches` reduces typical per-partition size, with more files and overhead. Default is 16. Hash partitioning is not sampling and should not change the reported statistics.

The implementation requires Polars >=2.0 and checks for `pl.PartitionBy`. Temporary summary work lives under the output folder and is cleaned up when the temporary-directory context exits. Basket/spell checkpoints are also temporary. Persisted prepared data, CSVs, LaTeX tables, and graphs remain.

10. Running, outputs, and verification

Legacy cleaning scripts execute at module top level and can write or replace large server datasets. Read their path constants and file dependencies before running them; importing them is not a harmless way to inspect functions. `summ_stats.py` and `recover_flavors.py` have guarded entry points.

From `final_draft/data_code`:

```
python3 summ_stats.py --data-dir /path/to/Data \
  --flavors /path/to/Latest_Flavor_2010.csv \
  --products-extra /path/to/Data/RMS/combined/products_extra.parquet \
  --output /path/to/Data/summary_outputs \
  --first-year 2006 --last-year 2015 --household-batches 16
```

`--no-figures` skips plotting while retaining summaries. To recreate HHI figures from saved household summaries alone:

```
python3 summ_stats.py --hhi-only --output /path/to/Data/summary_outputs
```

This requires the detailed `yogurt_household_flavors.csv` and also updates grouped HHI graphs when that grouped file exists. It does not reread purchases.

Output	Meaning
`full_panel_households.csv`	Household summaries from full purchase panel
`matched_yogurt_households.csv`	Household summaries after matched-sample price/two-week filtering
`{prefix}_{demographic}.csv/.tex`	Unweighted categorical distributions with missing group
`{prefix}_overview.csv/.tex`	Household counts, means, rates, and race percentages
`{prefix}_year_coverage.csv/.tex`	Exact annual sample coverage
`yogurt_price_coverage.csv/.tex`	Lines excluded by the matched-price condition
`recovered_flavor_reference.csv` and companion rows	Original-plus-supplemented descriptor reference
`detailed_flavor_reference.csv`	Resolved detailed UPC labels
`estimation_flavor_reference.csv`	Resolved three-group UPC labels
`yogurt_sample_audit.csv/.tex`	Detailed-pass purchase/label/quantity audit
`yogurt_flavor_shares.csv/.tex`	Package-weighted flavor shares
`yogurt_household_flavors.csv`	Household concentration and breadth

Output	Meaning
`yogurt_weekly_flavor_quantities.csv`	Sparse weekly flavor quantities
`yogurt_weekly_baskets.csv`	Basket totals, labels, gaps, eligibility, spell IDs
`yogurt_transition_counts.csv` / `yogurt_transition_probabilities.csv`	Eligible adjacent single-flavor transitions
`yogurt_single_flavor_spells.csv`	Consecutive single-flavor run lengths
`yogurt_dynamics.csv/.tex`	Dynamics overview
`flavor_hhi_histogram.pdf`, `flavor_hhi_cdf.pdf`	Equal-household concentration distributions
`flavor_shares.pdf`, `flavor_transitions.pdf`	Up-to-20-flavor visual summaries
`aggregated_flavors/`	Corresponding results for other/berry/plain
`yogurt_estimation_sample.parquet` under output	Full retained purchase columns with saved grouped flavor

The final export reopens the original master sample, filters years/positive prices/eligible households, drops an existing flavor column if necessary, and joins the exact grouped mapping. It refuses to overwrite the original source path. It preserves store and DMA fields not needed by demographic projection.

Synthetic regression checks can run without private data:

```
python3 -m unittest -v test_summ_stats_memory.py
```

Related summary tests live in `../est_code/test_summary_stats.py`. The memory checks compare outputs across partition counts and exercise preservation of original purchase metadata. They validate controlled examples; private-data coverage, price conventions, and join losses still require data audits.

When totals look surprising, trace the first stage where they change: source year coverage, trip matching, retail matching, price coverage, two-week selection, flavor matching, and finally estimator exclusions. Package shares, equal-household demographics, transition pairs, and spell means use different denominators. Their differences are often explained by their units rather than a calculation error.